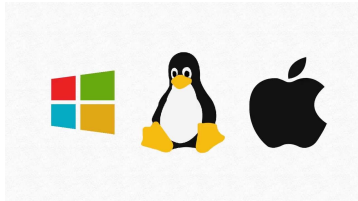


컴퓨터 시스템

File System



This material is reproduced based on
핵심 컴퓨터구조 및 운영체제 해설(최종언저, 예문사),
쉽게 배우는 운영체제(조성호, 한빛아카데미, 2018)
Copyrights are reserved by corresponding authors.

2-3 디스크 장치 관리

파티션

- 디스크를 논리적으로 분할하는 작업
- 파티션 하나에 하나의 파일 시스템이 탑재
- 하드디스크가 2개라면 첫 번째 하드디스크와 두 번째 하드디스크가 별도의 파티션으로 보임
- 대용량 하드디스크의 경우 하나로 사용하기보다 여러 개로 나누어 사용하면 관리하기가 편함
- 여러 개의 하드디스크를 하나의 파티션으로 통합하여 사용하기도 함

마운트mount

- 유닉스 운영체제에서 여러 개의 파티션을 하나로 통합하는 기능

2-3 디스크 장치 관리

파티션



그림 10-18 디스크와 파티션의 관계

2-3 디스크 장치 관리

포매팅

- 디스크에 파일 시스템을 탑재하고 디스크 표면을 초기화하여 사용할 수 있는 형태로 만드는 작업
- 빈 저장장치에 파일 테이블을 탑재

파일 A	1, 3, 9
파일 B	4, 2
파일 C	13
파일 D	15, 4
파일 E	23, 7

그림 10-19 파일 테이블

빠른 포매팅과 느린 포매팅

- 하드디스크를 처음 사용할 때는 느린 포매팅을 하고, 하드디스크를 사용하다가 다시 포매팅을 해야 할 때는 빠른 포매팅을 하면 됨
 - 빠른 포매팅 : 데이터는 그대로 둔 채 파일 테이블을 초기화하는 방식
 - 느린 포매팅 : 디스크에 파일 시스템을 탑재하고 디스크 표면을 초기화하는 방식

2-3 디스크 장치 관리

조각모음

- 하드디스크에 조각이 많이 생기면 큰 파일이 여러 조각으로 나뉘어 저장되고 이를 읽기 위해 하드디스크의 여러 곳을 돌아다녀야 하기 때문에 성능 저하로 이어짐
- 하드 디스크는 주기적으로 조각 모음을 해주어야 함

SSD X

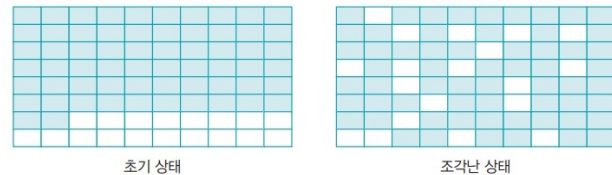


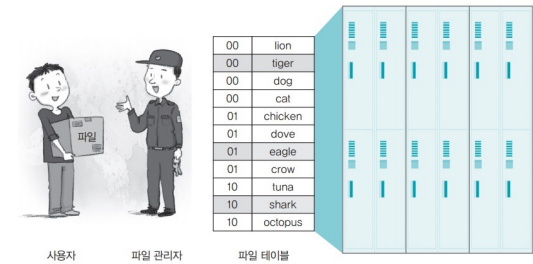
그림 10-20 초기 상태와 조각난 상태

© 2021 Computer System

1-1 파일 시스템의 개요

파일 시스템

- 사용자가 직접 파일을 보관하는 대신 파일 관리자를 두어 저장 장치의 관리를 맡기는 시스템
- 파일 관리자가 파일 테이블을 사용하여 파일을 관리
- 사용자가 특정 파일에 접근하려면 파일 관리자로부터 파일에 접근할 수 있는 권한(키)을 획득해야 함



© 2021 Computer System

1-1 파일 시스템의 개요

표 11-1 파일 시스템의 기능

기능	설명
파일 구성	사용자의 요구에 따라 파일과 디렉터리를 만든다.
파일 관리	파일 생성, 수정, 삭제 등의 관리를 하고, 수시로 조각 모음을 하여 사용자가 파일에 빨리 접근할 수 있도록 한다.
접근 권한 관리	다른 사용자로부터 파일을 보호하기 위해 접근 권한을 관리한다.
접근 방법 제공	파일을 읽고 쓰고 실행할 수 있도록 사용자에게 접근 방법을 제공한다.
무결성 보장	파일의 내용이 손상되지 않도록 무결성을 보장한다.
백업과 복구	사고로부터 파일을 보호하기 위해 백업과 복구 작업을 한다.
암호화	파일을 암호화하여 악의적인 접근으로부터 파일을 보호한다.

© 2021 Computer System

1-1 파일 시스템의 개요

블록과 파일 테이블

- 블록은 저장장치에서 사용하는 가장 작은 단위로, 한 블록에 주소 하나가 할당
- 블록은 여러 개의 섹터로 구성되며 블록의 크기는 시스템마다 다름
- 블록 크기를 작게 설정하면 내부 단편화 현상이 줄어들어 저장장치를 효율적으로 쓸 수 있지만, 파일이 여러 블록으로 나뉘어 파일 입출력 속도가 느려짐
- 큰 파일을 많이 사용할 때는 블록 크기를 크게 잡는 것이 좋음



그림 11-2 윈도우의 블록 크기 설정

© 2021 Computer System

1-2 파일 분류와 확장자

■ 파일 분류

- 실행 파일 : 운영체제가 메모리로 가져와 CPU를 이용하여 작업을 하는 파일
- 데이터 파일 : 실행 파일이 작업하는 데 필요한 데이터를 모아놓은 파일

표 11-2 파일의 분류와 확장자

파일	확장자	설명
실행 파일	exe, com	유닉스에는 실행 파일 확장자가 없음
소스코드 파일	c, cpp, pas, a, java	다양한 소스코드의 확장자
라이브러리 파일	lib, a, dll	소스코드를 위한 라이브러리의 확장자
배치 파일	bat, sh, csh	초기 배치 파일의 확장자
문서 파일	txt, doc, hwp, pdf, ps 등	문서 데이터 파일의 확장자
동영상 파일	avi, asf, mkv, mov, rmv	동영상 데이터 파일의 확장자
음악 파일	wav, mp3, ogg, flc, aac	음악 데이터 파일의 확장자
이미지 파일	bmp, gif, jpg, tiff, png	이미지 데이터 파일의 확장자
압축 파일	rar, zip, arc, al	압축 파일의 확장자

© 2021 Computer System

1-3 파일 이름과 연결 프로그램

■ 파일 이름

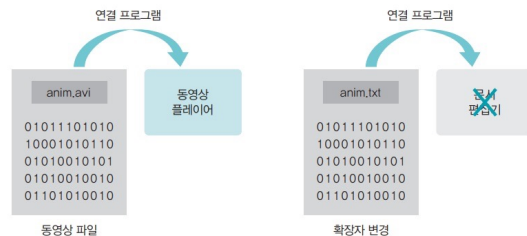
- 사용자 입장에서 어떤 응용 프로그램이 사용하는 데이터인지 구분할 필요가 있음. 예를 들어 같은 문서파일이라 할지라도 아래아한글용 문서 파일인지 MS워드용 문서 파일인지 구분할 필요가 있음
- 데이터의 종류를 구분하기 위해 확장자를 사용
- 파일 확장자는 3자 이하가 대부분이지만 4자나 5자로 된 확장자도 있음
- 파일 이름에 마침표(.)를 여러 번 사용할 수 있는데, 이때는 마지막 마침표 다음 글자를 확장자로 인식
- 파일 이름은 현재 경로 이름을 포함하여 최대 255자
- 파일 이름에는 영문자, 숫자, 불임표(-), 밑줄(_), 마침표(.)를 주로 사용

© 2021 Computer System

1-3 파일 이름과 연결 프로그램

■ 연결 프로그램

- 데이터 파일을 더블클릭하면 해당 파일을 사용하는 응용 프로그램이 실행되는데 이러한 프로그램을 연결 프로그램이라고 함
- 윈도우에서 데이터 파일에 연결된 응용 프로그램을 바꾸고 싶을 때는 연결 프로그램을 변경하면 됨
- 확장자를 바꾼다고 해서 내용이 바뀌지 않음



© 2021 Computer System

1-4 파일 속성

■ 파일 속성의 종류

표 11-3 파일 속성의 종류

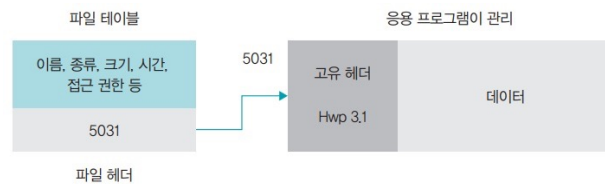
속성	특징
name	파일의 이름
type	파일의 종류
size	파일의 크기
time	파일의 접근 시간
location	파일의 위치
accessibility	파일의 접근 권한
owner	파일의 소유자

© 2021 Computer System

1-4 파일 속성

■ 파일 헤더와 고유 헤더

- 파일 헤더 : 파일 테이블에서 관리하며 파일의 이름, 종류, 크기, 시간, 접근 권한 등과 같은 일반적인 내용과 파일이 저장장치의 몇 번째 블록에 있는지에 대한 정보를 가지고 있음
- 고유 헤더 : 데이터 파일에는 응용 프로그램이 필요로 하는 고유 헤더가 따로 달려 있는데 파일의 버전 번호, 크기, 특수 정보 등 응용 프로그램이 필요로 하는 정보가 있음



© 2021 Computer System

1-5 파일 작업의 유형

표 11-4 파일 자체를 변경하는 작업

작업	설명	작업	설명
open	파일을 연다.	copy	파일을 복사한다.
close	파일을 닫는다.	rename	파일의 이름을 변경한다.
create	새로운 파일을 생성한다.	list	파일을 나열한다.
remove	파일을 이동한다.	search	파일을 찾는다.

표 11-5 파일 내용을 변경하는 작업

작업	설명	작업	설명
open()	파일을 연다.	write()	파일에 새로운 내용을 쓴다.
create()	새로운 파일을 생성한다.	update()	파일 내용 중 일부를 변경한다.
close()	파일을 닫는다.	insert()	파일에 새로운 내용을 추가한다.
read()	파일 내용을 읽는다.	delete()	파일 내용 중 일부를 지운다.

© 2021 Computer System

1-6 파일 구조

■ 순차 파일 구조sequential file structure

- 파일 내용이 하나의 긴 줄로 늘어선 형태
- 장점
 - 모든 데이터가 순서대로 기록되기 때문에 저장 공간에 낭비되는 부분이 없음
 - 구조가 단순하여 테이프는 물론 플로피디스크나 메모리를 이용한 저장장치에도 적용할 수 있음
 - 순서대로 데이터를 읽거나 저장할 때 매우 빠르게 처리됨
- 단점
 - 파일에 새로운 데이터를 삽입하거나 삭제할 때 시간이 많이 걸림
 - 특정 데이터로 이동할 때 직접 접근이 어렵기 때문에 앞에서부터 순서대로 움직여야 하기 때문에 데이터 검색에 적당하지 않음



그림 11-10 순차 파일 구조

© 2021 Computer System

1-6 파일 구조

■ 인덱스 파일 구조index file structure

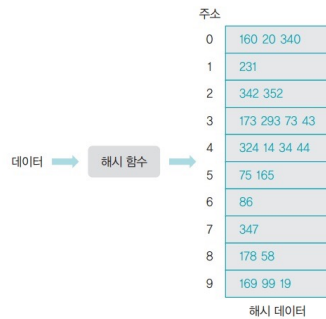
- 순차 파일 구조에 인덱스 테이블을 추가하여 순차 접근과 직접 접근이 가능
- 현대의 파일 시스템은 인덱스 파일 구조로, 파일을 저장할 때는 순차 파일 구조로 저장하고 파일에 접근할 때는 인덱스 테이블을 보고 원하는 파일에 직접 접근



© 2021 Computer System

1-6 파일 구조

- 직접 파일 구조direct file structure
 - 저장하려는 데이터의 특정 값에 어떤 관계를 정의하여 물리적인 주소로 바로 변환하는 파일 구조
 - 특정 함수를 이용하여 직접 접근이 가능한 파일 구조
 - 이때 사용하는 함수를 해시 함수hash function라고 함



© 2021 Computer System

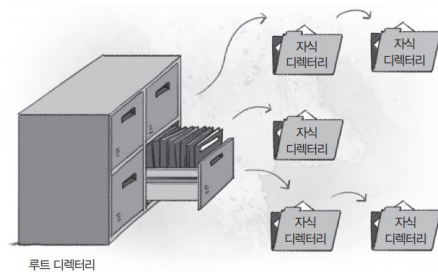
1-6 파일 구조

- 직접 파일 구조의 장점
 - 해시 함수를 이용하여 주소를 변환하기 때문에 데이터 접근이 매우 빠름
- 직접 파일 구조의 단점
 - 직접 파일 구조를 제대로 작성하려면 해시 함수의 선정이 매우 중요한데, 전체 데이터가 고르게 저장될 수 있는 해시 함수를 찾기가 어려움
 - 해시 함수를 잘 찾았다고 하더라도 저장 공간이 낭비되는 문제도 있음

© 2021 Computer System

2-1 디렉터리의 개념

- 디렉터리**
 - 관련 있는 파일을 하나로 모아놓은 곳
 - 1개 이상의 자식 디렉터리를 가질 수 있고, 또한 1개 이상의 파일을 가질 수 있음
 - 하나의 디렉터리에는 여러 개의 파일과 자식 디렉터리가 존재



© 2021 Computer System

2-1 디렉터리의 개념

- 디렉터리의 계층 구조
 - 디렉터리는 여러 층으로 구성할 수 있음
 - 루트 디렉터리root directory** : 최상위에 있는 디렉터리

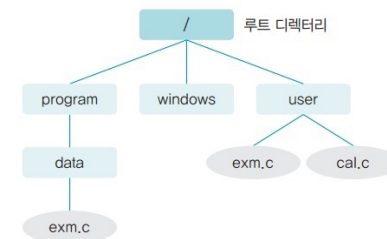


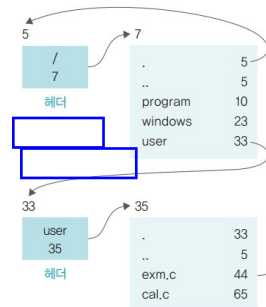
그림 11-14 디렉터리 계층 구조의 예

© 2021 Computer System

2-2 디렉터리 파일

■ 디렉터리 파일

- 디렉터리도 파일임
- 일반 파일에는 데이터가 담기고 디렉터리에는 파일 정보가 담김
- 디렉터리 헤더에는 디렉터리의 이름, 만든 시간, 접근 권한 등의 정보가 기록됨

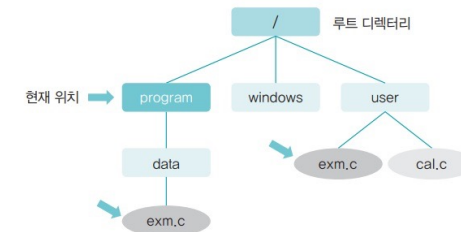


© 2021 Computer System

2-3 경로

■ 경로

- 파일이 전체 디렉터리 중 어디에 있는지를 나타내는 정보
- 한 디렉터리에는 같은 이름을 가진 파일이 존재할 수 없지만, 서로 다른 디렉터리에는 같은 이름의 파일이 존재할 수 있음
- 절대 경로 : 루트 디렉터리를 기준으로 파일의 위치를 나타내는 방식
- 상대 경로 : 현재 있는 위치를 기준으로 파일의 위치를 표시하는 방식



© 2021 Computer System

2-4 디렉터리 구조

■ 1단계 디렉터리 구조

- 루트 디렉터리에 새로운 디렉터를 만들 수 있지만 디렉터리 안에 자식 디렉터를 만들 수는 없음

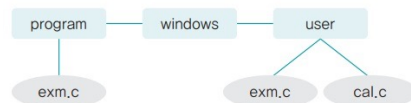


그림 11-17 1단계 디렉터리 구조의 예

© 2021 Computer System

2-4 디렉터리 구조

■ 다단계 디렉터리 구조

- 단계 확장에 제약이 없고 디렉터리에 파일과 디렉터를 둘 다 저장할 수 있음

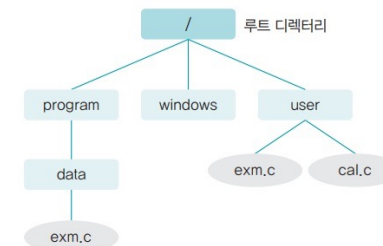


그림 11-18 다단계 디렉터리 구조의 예

© 2021 Computer System

2-4 디렉터리 구조

- 바로가기 링크를 포함한 디렉터리 구조
 - 원래 디렉토리 구조는 트리(tree) 구조였으나 '바로가기'로 인하여 그래프 구조로 바뀜*

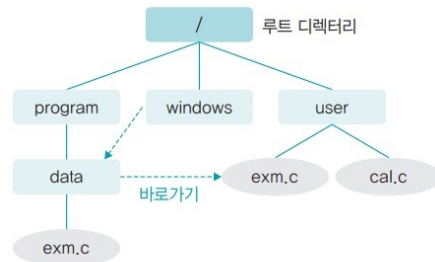


그림 11-20 그래프 디렉터리 구조

* 사이클이 없는 그래프를 트리(tree)라 함.

© 2021 Computer System

2-5 마운트

- 마운트
 - 유닉스에서 여러 개의 파티션을 통합하는 명령어
 - 2개의 파티션을 하나의 파일 시스템으로 만든 예

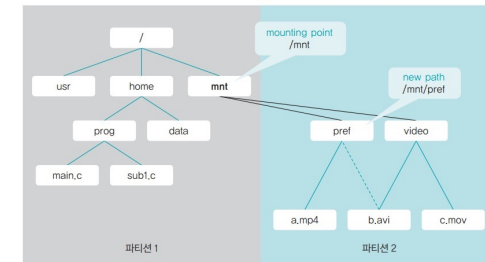


그림 11-23 유닉스의 마운트

© 2021 Computer System

3-1 연속 할당과 불연속 할당

- 파일 내 블록을 연결하는 방식
 - 연속 할당 방식
 - 파일을 구성하는 데이터를 디스크상에 연속적으로 배열하는 간단한 방식
 - 파일을 저장하거나 삭제하다 보면 빈 공간이 생기는데, 디스크에 남은 공간 중 파일의 크기와 맞는 연속된 공간이 없을 때는 연속 할당이 불가능하기 때문에 실제로 사용되지 않음
 - 불연속 할당 방식
 - 비어 있는 블록에 데이터를 분산하여 저장하고 이에 관한 정보를 파일 시스템이 관리하는 방식
 - 연결 리스트를 이용한 연결 할당과 인덱스를 이용한 인덱스 할당이 대표적

3-1 연속 할당과 불연속 할당

- 연결 할당
 - 파일에 속한 데이터를 연결 리스트로 관리하는 방식
 - 파일 테이블에는 시작 블록에 대한 정보만 저장하고, 나머지 데이터는 시작 블록부터 연결하여 저장
 - 체인으로 연결한 것처럼 보여서 체인 할당(chained allocation)이라고도 함
 - 윈도우의 FAT(File Allocation Table)이 이 방식을 사용

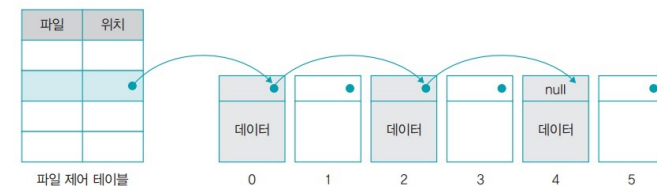


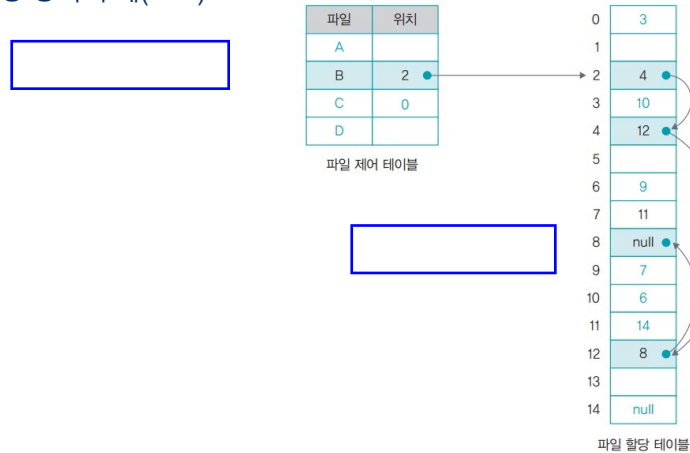
그림 11-24 연결 할당 방식

© 2021 Computer System

© 2021 Computer System

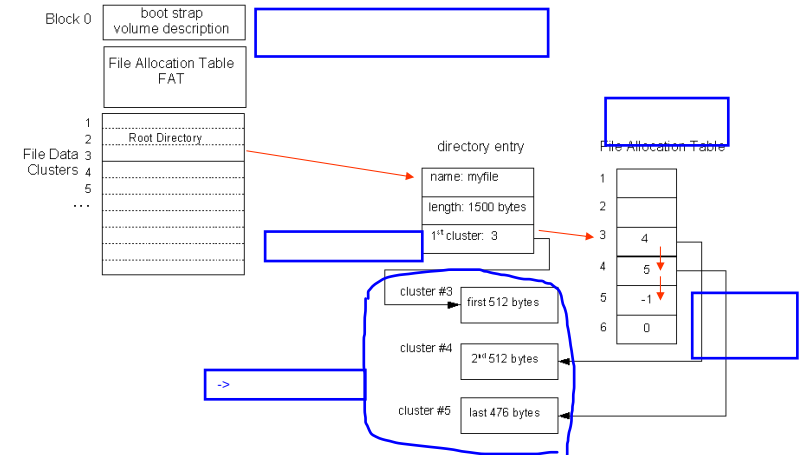
3-1 연속 할당과 불연속 할당

연결 할당 방식의 예(FAT)



© 2021 Computer System

Windows(DOS) FAT

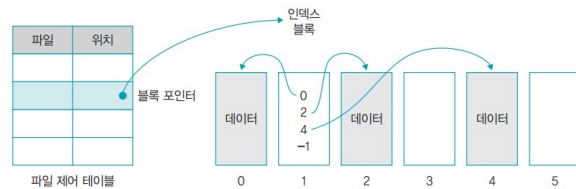


© 2021 Computer System

3-1 연속 할당과 불연속 할당

인덱스 할당

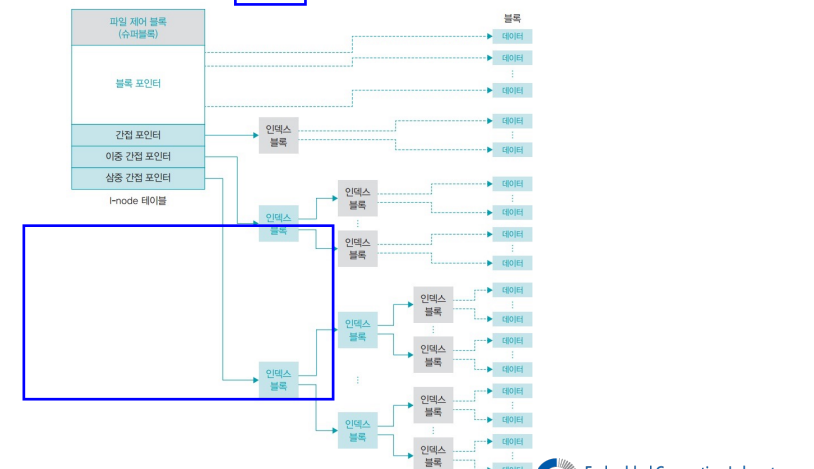
- 테이블의 블록 포인터가 데이터 블록을 연결하는 것이 아니라, 데이터의 인덱스를 담고 있는 인덱스 블록을 연결
- 인덱스 블록은 실제 데이터의 위치에 관한 정보를 순서대로 보관
- 테이블이 꽉 차서 더 이상 데이터를 연결할 수 없을 때는 인덱스 블록을 연결하는 간접 인덱스 블록(indirect index block)을 만들면 테이블을 무한히 확장할 수 있음
- 유닉스의 I-node가 이 방식을 사용



© 2021 Computer System

3-1 연속 할당과 불연속 할당

인덱스 할당 방식의 예(I-node)



© 2021 Computer System

3-2 디스크의 빈 공간 관리

빈 공간 리스트

- 파일 시스템은 디스크의 내부 단편화를 줄이고 빈 공간을 효율적으로 관리하기 위해 빈 블록의 정보만 모아놓은 빈 공간 리스트를 유지
- 파일 시스템에서는 파일 테이블의 헤더를 삭제하고 사용했던 블록을 빈 공간 리스트에 등록하는 것을 파일이 삭제된 것으로 간주
- 어떤 데이터를 지우고 새로운 데이터를 디스크에 넣을 때 방금 지워진 블록에 할당되는 것이 아니라 리스트에 있던 블록 중 맨 앞에 있는 블록에 할당

© 2021 Computer System

3-2 디스크의 빈 공간 관리

빈 공간 리스트

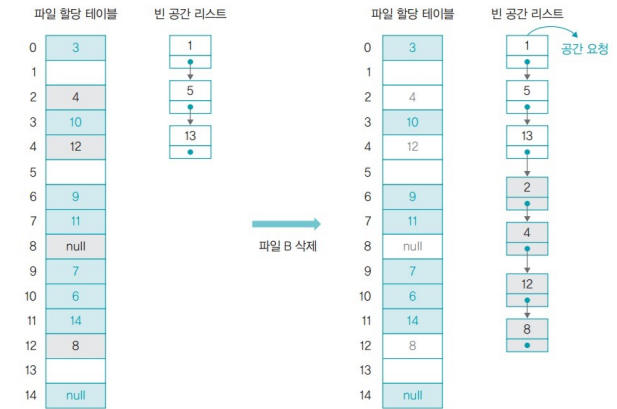


그림 11-29 빈 공간 리스트의 변화

© 2021 Computer System

Summary



Image from <http://www.promotionalpro.com/blog/>

**Don't Guess....
Just ASK !**

© 2021 Computer System